

AWS Lambda

1. AWS Lambda:

AWS Lambda is a serverless compute service.

This means:

- You do NOT create or manage servers.
- You only write and upload your code.
- AWS automatically runs your code only when needed.

Simple Example:

Instead of buying a generator and keeping it running, Lambda is like paying for electricity only when you switch ON a light.

So, you pay ONLY when your function runs.

No charges if it's idle.

2. Why is Lambda Used?

AWS Lambda is popular because it:

Benefits	Meaning
 Cost-efficient	No servers running 24/7
 Auto-scaling	Handles 1 → 100,000 requests automatically
 No server management	Focus only on code
 Faster deployment	No OS or software installation

3. Important Concepts

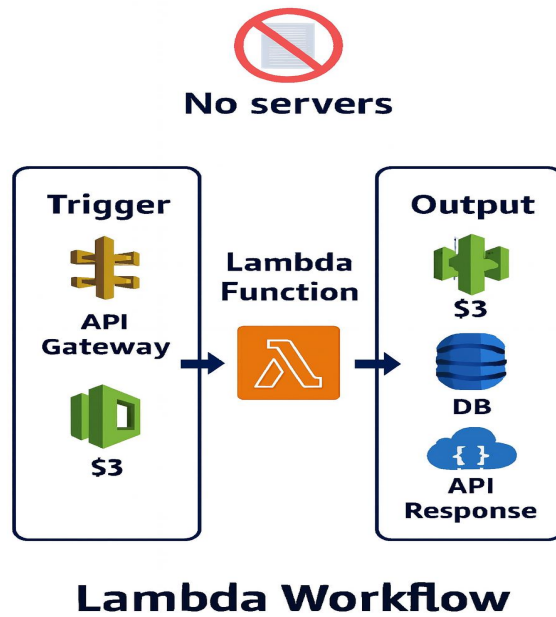
Concept	Meaning	Example
Function	Your code that Lambda executes	Python script
Handler	Entry point of execution	<code>lambda_handler(event, context)</code>
Event	Input data	S3 upload, API request
Trigger	Who calls Lambda	S3, API Gateway, SQS
Execution Role	IAM permission for Lambda	Allow read/write to S3 bucket
Layer	Add libraries	NumPy, pandas
Timeout	Max execution time	Default: 3 sec → Max: 15 min

4. How Lambda Works

Example: Image uploaded → Lambda resizes it.

1. Developer uploads code to Lambda
2. Connects an S3 event trigger
3. User uploads an image
4. S3 triggers Lambda automatically
5. Lambda resizes the image
6. Stores output in another bucket

 No server setup needed — AWS handles everything.



5. Supported Languages

- ✓ Python, Node.js, Java, Go, Ruby, .NET
 - ✓ Custom runtime using Docker
-

6. Real-World Use Cases

Category	Example
File Processing	Image resize, file conversion
API Backend	REST APIs using API Gateway
Automation	Stop unused EC2
Scheduler	Run daily/weekly tasks
Streaming	Process DynamoDB/Kinesis events
Security	Auto-remediation alerts

7. Pricing

You pay for:

- Number of requests
- Duration × Memory assigned

Free tier every month:

- 1 Million requests free
- 400,000 GB-seconds free

Example: If code runs 200ms with 128MB memory → mostly free.

8. Lambda can be triggered by:

- API Gateway → API calls
 - S3 → File uploads
 - CloudWatch → Scheduled jobs
 - SNS/SQS → Messages
 - DynamoDB Streams → DB changes
 - Kinesis → Streaming
 - Step Functions → Workflows
-

9. Lambda Execution Lifecycle

Phase	Meaning
INIT	Environment setup (cold start)
INVOKE	Handler executes → Billing applies
WAIT/WARM	Container stays warm for reuse

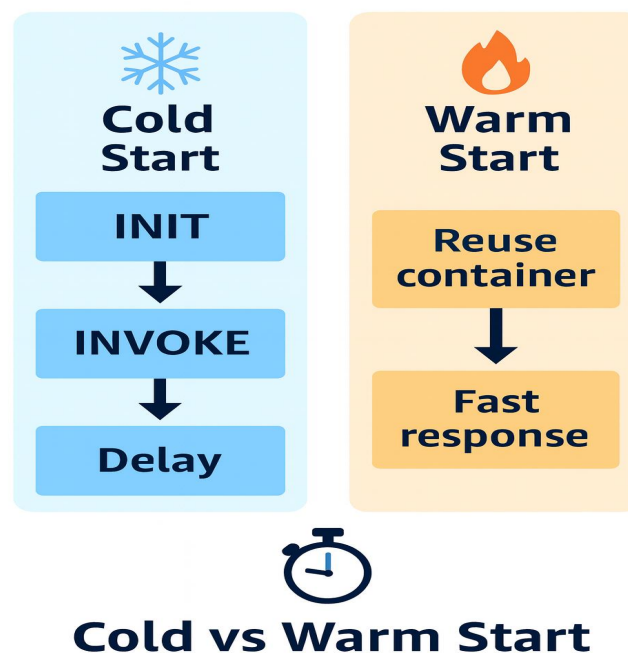
Warm start = faster

Cold start = slight delay (first run)

10. Cold Start vs Warm Start

Type	Speed	When Happens
Cold Start	Slow (due to setup)	First time or after long idle
Warm Start	Fast	Reusing previous container

To avoid cold starts → use Provisioned Concurrency.



11. Error Handling

- Synchronous → Caller gets error response
- Asynchronous → Lambda retries 2 times
- Failed events → Send to DLQ (SQS/SNS)

12. Security in Lambda

- ✓ IAM Execution Role
 - ✓ Resource policies
 - ✓ Environment variables encrypted using KMS
 - ✓ Can run inside VPC if needed
-

13. Monitoring & Logging

Tool	Purpose
CloudWatch Logs	Logs and errors
CloudWatch Metrics	Duration, errors, throttles
AWS X-Ray	Tracing & debugging

14. Deployment Methods

- Upload .zip
 - AWS CLI
 - Terraform / SAM / CDK
 - Docker image via ECR
-

15. Best Practices Summary

- ✓ Keep functions focused (single responsibility)
- ✓ Use environment variables
- ✓ Use Lambda layers for large libraries
- ✓ Increase memory if function slow

- ✓ Use DLQ / Destination for failures
 - ✓ Enable Provisioned Concurrency for APIs
-

16. Lambda Invocation Types

Type	Retry?	Example
Synchronous	No	API calls
Asynchronous	Yes	S3, SNS
Polling (ESM)	Yes	SQS, DynamoDB Streams

17. Important Limits

Limit	Value
Max runtime	15 mins
Memory	128MB → 10GB
ZIP size	50MB upload / 250MB extracted
Environment variables	4KB
Default concurrency	1000

18. Destinations (For Async Events)

Event Type	Destination
Success	SQS, SNS, Lambda, EventBridge
Failure	SQS, SNS, Lambda, EventBridge

19. Versions & Aliases

- Version → Fixed code snapshot
- Alias → Name pointing to version (dev/test/prod)
- Supports traffic shifting for safe rollout.

20. Additional Advanced Concepts

- Event Filtering
- EFS support
- Docker runtime
- RDS Proxy for database connections
- Step Functions → Complex workflows